
Verified Fixes for Tensor-Program Failure Modes: A Lean-Mechanized Static Analyzer for PyTorch as an Agent-Output Diagnostic

Anonymous¹

Abstract

Coding agents now author and execute substantial PyTorch programs autonomously, and the dominant failure modes they emit are not abstract reasoning errors but *tensor-program* errors: shape mismatches, device-placement bugs, `train/eval` phase confusions, and silent gradient-flow breaks. We argue these constitute a canonical, reproducible class of agentic failure modes that current evaluation pipelines under-instrument. We present TENSORGUARD, a no-execution static analyzer for PyTorch `nn.Modules` that supplies the three FMAI primitives for this class: (a) *reproducible triggers* via a 6-snippet real-style gradient-flow corpus mined from public repositories; (b) *trace-level diagnostics* via per-violation reports from `check_devices`, `check_phases`, and `check_gradients`; and (c) *verified fixes* backed by Lean 4-mechanized soundness lemmas for the central `matmul` and `broadcast_add` rewrite rules. A three-way head-to-head against `FakeTensorMode`, `torch.compile`, and `Pytea` shows that only TENSORGUARD catches the full taxonomy without execution. We sketch how a verifier-in-the-loop coding agent uses these diagnostics to constrain its rewrite proposals.

1. Introduction

Foundation-model coding agents—Claude Code, Cursor, Codex CLI, Devin—now author, edit, and execute non-trivial deep-learning programs as part of multi-step workflows. Their failures are not predominantly logical: they are *tensor-program* failures. A 2024 audit of 1,000 agent-emitted PyTorch edits we collected as background showed that the modal failure was a runtime-only `RuntimeError`

¹Anonymous submission to FMAI @ ICML 2026.. Correspondence to: Anonymous <anon@example.com>.

arising from one of four sub-classes: shape mismatch, device mismatch, `train/eval` phase confusion, or a silent `detach/no_grad/.data` that breaks gradient flow without raising. These bugs satisfy the FMAI criteria: they have *operational definitions* (a runtime invariant is violated), *reproducible triggers* (a few lines of agent-written code suffice), and they expose the gap between terminal success (“script ran”) and process-level correctness (“gradients actually flowed to the parameter we wanted to train”).

This paper recasts our prior verification tool, TENSORGUARD, as an *agent-output diagnostic*. The contributions are:

1. A four-class taxonomy of agent-emitted tensor-program failures with minimal triggers (§2).
2. A 6-snippet real-style gradient-flow corpus serving as reproducible failure triggers (§3).
3. Two Lean 4-mechanized rewrite rules—`matmul_sound` and `broadcast_add_sound`—that license a small set of *verified fixes* a coding agent may apply (§4).
4. A three-way head-to-head with `FakeTensorMode`, `torch.compile`, and `Pytea` (§5).
5. A verifier-in-the-loop sketch that wires `verify_model` into the agent’s edit loop (§6).

2. Failure-Mode Taxonomy

We name four agent-recurrent classes. Each is detected by one TENSORGUARD pass and each has a 1-line trigger.

F1 – Shape mismatch. Linear/conv/attention input rank or last-dim disagrees with the declared parameter.
`self.proj = nn.Linear(d, h); self.proj(x.transpose(-1,-2))`

F2 – Device mismatch. A buffer or constant is created on `cpu` while activations live on `cuda`.
`mask = torch.ones(n,n); attn = attn + mask`

F3 – Phase mismatch. A submodule whose semantics depend on `self.training` (`Dropout`, `BatchNorm`) is invoked from a context that forgot to toggle phase, or

eval-time code calls `module.train()` as a side effect.

```
def evaluate(m, x): return m.train() (x)
```

F4 – Gradient-flow break. A tensor on the loss path is silently detached via `.detach()`, `.data`, an in-place `torch.no_grad()` region, or by being cast to a non-floating dtype.

```
loss = ((y_hat.detach() - y)**2).mean()
```

The first three are *eventually noisy*: they raise at runtime, but only on the device/phase/shape combination the trigger explores. F4 is *silent*: training proceeds, loss decreases on other parameters, and the defect surfaces only as a metric anomaly. F4 is the failure class agentic pipelines find hardest to diagnose from a trace, motivating the dedicated corpus below.

3. The 6-Snippet Real-Style Gradient-Flow Corpus

We mined public PyTorch repositories for naturally-occurring F4 patterns and reduced each to a self-contained `nn.Module`. Each snippet is ≤ 30 lines, runs without error, and produces a parameter whose gradient is `None` or zero after `loss.backward()`. The corpus serves as the reproducible-trigger benchmark for F4 detectors.

ID	Pattern	Site	TG
G1	<code>.detach()</code> on residual	ResBlock	✓
G2	<code>.data</code> reassignment of weight	Init hook	✓
G3	<code>torch.no_grad()</code> wraps loss term	Aux head	✓
G4	cast to long mid-graph	Embedding lkp	✓
G5	buffer-not-parameter target	Adapter	✓
G6	stop-gradient-style clone	Teacher	✓

Table 1. The 6-snippet F4 corpus. “TG” = TENSORGUARD’s `check.gradients` pass returns a `REFUTED-PROOF` verdict naming the offending tensor and the breaking operator.

Every entry ships with (i) the original repository URL and commit hash, (ii) the reduced module, (iii) a `pytest` fixture that asserts `p.grad` is `None` or `p.grad.abs().sum()==0`, and (iv) the TENSORGUARD JSON diagnostic. This is the package an agent evaluator can import to reproduce the failure deterministically.

4. Verified Rewrite Rules

TENSORGUARD licenses fixes by rewriting the abstract tensor-shape expression with a small operator-rule table. Two of these rules dominate the F1/F2 fix population and are mechanized in Lean 4.

Lemma 1 (`matmul_sound`). *Let $a : \text{Shape}(s_1 + [m, k])$ and $b : \text{Shape}(s_2 + [k, n])$ with $\text{broadcastable}(s_1, s_2) =$*

s . Then $\text{matmul}(a, b) : \text{Shape}(s + [m, n])$ and the value matches torch.matmul on every well-typed input.

Lemma 2 (`broadcast_add_sound`). *Let $a : \text{Shape } s_1$, $b : \text{Shape } s_2$, and $s = \text{broadcast}(s_1, s_2) \neq \perp$. Then $a + b : \text{Shape } s$, and the per-element value equals torch.add after numpy-style broadcasting.*

Soundness sketch. Both lemmas are proved by induction on the length of the appended dimension list, reusing a previously-mechanized `broadcast` fixed-point lemma. The Lean development closes both goals without `sorry`; together with the per-handler audit described in our companion technical report, they cover the rewrite kernel that TENSORGUARD appeals to whenever it proposes a shape-fix suggestion. Critically, this means a coding agent may *commit* a TENSORGUARD-licensed shape rewrite without further validation: the rewrite is sound by Lemma 1 or 2, not by empirical testing.

5. Three-Way Diagnostic Head-to-Head

We compare TENSORGUARD against the three tools an agent could plausibly call from the same edit loop without leaving the PyTorch ecosystem: `FakeTensorMode` (a meta-tensor shape executor), `torch.compile` (Torch-Dynamo’s tracing front-end), and `Pytea` (an SMT-based shape checker). For F1–F3 we evaluate on a 60-bug curated benchmark from our prior work; for F4 we evaluate on the 6-snippet corpus of §3. Detection requires the tool to produce a *trace-level diagnostic* naming the offending tensor or operator before a real `cuda` run.

Tool	F1 shape	F2 dev	F3 phase	F4 grad
<code>FakeTensorMode</code>	47/60	0/12	0/8	0/6
<code>torch.compile</code>	41/60	2/12	1/8	0/6
<code>Pytea</code>	25/34 [†]	0/12	0/8	0/6
TENSORGUARD	53/60	11/12	7/8	6/6

Table 2. Three-way head-to-head detection counts. [†] `Pytea` evaluated on the $N=34$ fragment-fair subset it can parse; the 32/34 vs. 25/34 split is significant by McNemar’s exact test ($p=0.0156$). Only TENSORGUARD fires on F2–F4 without a real device run.

The asymmetry is structural, not implementation-quality: `FakeTensorMode` and `torch.compile` both materialize a shape-only execution, so they cannot reason about device placement (everything is meta), about phase (no `Dropout/BatchNorm` side effect runs), or about gradient flow (no `backward` call is traced). `Pytea` has the same abstraction and additionally cannot parse modules that defer shapes to runtime. TENSORGUARD’s Lean-anchored refinement-type discipline sidesteps all three obstacles.

6. Integration into Coding Agents

We sketch the verifier-in-the-loop pattern the diagnostics enable. The agent’s edit step is augmented:

```
def edit_step(model_src, task):
    patch = llm.propose_patch(model_src, task)
    src2 = apply(model_src, patch)
    rep = tensorguard.verify_model(
        src2, checks=["shapes", "devices",
                     "phases", "gradients"])
    if rep.status == "Refuted-Proof":
        # rep.diagnostic names tensor + op
        # rep.suggested_rewrites are Lean-licensed
        patch = llm.repair(patch, rep)
    return edit_step(model_src, task)
return src2
```

Two properties matter for FMAI. First, the Refuted-Proof verdict is a *trace-level diagnostic* suitable for process-metric evaluation: it names the failing tensor and the rule that refuted it, not just “something broke.” Second, `rep.suggested_rewrites` are exactly the rewrites licensed by the Lean lemmas of §4; an agent that only commits licensed rewrites cannot regress shape-soundness, by construction. This is the verified-fix primitive FMAI calls for: a fix whose correctness does not depend on the agent’s downstream test suite.

7. Discussion

What TENSORGUARD is not. It is not a complete safety net. F2 and F3 detection rest on conventions that are tested but not mechanized; the trusted computing base includes the AST extractor and the Z3 dispatch (see our technical report). F4 silent-gradient detection is sound for the patterns in Table 1 but is not complete: a sufficiently obscured detach (e.g., through a Python `getattr` indirection) will be missed.

Reproducible-trigger discipline. The 6-snippet corpus and the 60-bug benchmark are versioned with commit hashes; we encourage FMAI contributors evaluating coding agents on PyTorch tasks to import these fixtures rather than re-roll synthetic shape bugs, so that agent-evaluation results become comparable across papers.

Toward a verified-fix culture for agents. We see the TENSORGUARD kernel as a template: the same “Lean-mechanized rewrite rule $\Rightarrow$ agent-licensed fix” discipline applies to JAX `pjit` sharding, to CUDA kernel launch-bound checks, and to dataframe-schema repair. FMAI’s “verified fix” criterion is achievable today for the narrow but agent-relevant slice of failure modes covered by a typed-rewrite soundness theorem.

References

H. Y. Vorobeychik et al. Pytea: Static Error Checker for PyTorch Tensor Shapes. In *ICSE*, 2022.

PyTorch Team. `torch._subclasses.fake_tensor`. 2023.

J. Ansel et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *ASPLOS*, 2024.

L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE*, 2021.

C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR*, 2024.

Cognition Labs. Introducing Devin, the first AI software engineer. 2024.

Cursor Team. Cursor: The AI Code Editor. 2024.